

Where EasyClaim stands, what we built, and how we talk about it

A single page for the whole team. It answers the question "are we done?" honestly, explains the security and quantum work in plain words, and gives you the pitch, the demo, and the answers judges will push on.

213

Worker security tests passing on `main`, plus 1 pre-existing todo

26

Python tests passing for the quantum experiment

0

npm audit vulnerabilities and 0 TypeScript errors on `main`

Are we done? Cyber: yes for what is on main. Quantum: yes, honestly scoped.

Everything below was verified against the GitHub `main` branch at commit `f1b2374` on 26 September 2026, after Phase 3 merged by running typecheck, the full test suite, the audit, and the Python tests on a fresh checkout.

PHASE	TRACK	WHAT IT DELIVERS	STATUS	EVIDENCE
0 Foundation	Cyber	Ownership checks, RBAC, state machine, strict validation, append-only audit, headers, limits	● LIVE ON MAIN	85 original tests, all still passing inside the 190
1 Identity and tenants	Cyber	Real JWT authentication, actor with tenant, cross-	● LIVE ON MAIN	183 tests, live attack log <code>PHASE_01_LIVE_ATTACKS.log</code>

PHASE	TRACK	WHAT IT DELIVERS	STATUS	EVIDENCE
		tenant isolation, insurer endpoints		
2 Evidence and OCR	Cyber	Private storage, hashes, untrusted-document boundary	● NOT ON MAIN	Teammate's machine. No remote branch seen yet. Migration number 0004 reserved.
3 Decision and payout	Cyber	Insert-only decision records, simulated payout protection, idempotency	● LIVE ON MAIN	PR #2 merged. Migration 0005, 213 tests, live demo log PHASE_03_LIVE_DEMOS.log
4 Quantum screening	Quantum	One-class quantum-kernel anomaly signal vs classical baseline, read-only integration	● LIVE ON MAIN	26 Python tests, 7 new Worker tests, live demo log PHASE_04_LIVE_DEMO.log
5 Post-quantum signatures	Quantum	ML-DSA signature over the decision bundle, tamper demo	● NOT STARTED	Planned in master plan §11. Phase 3 already reserved the column for it.

What "done" means for you personally

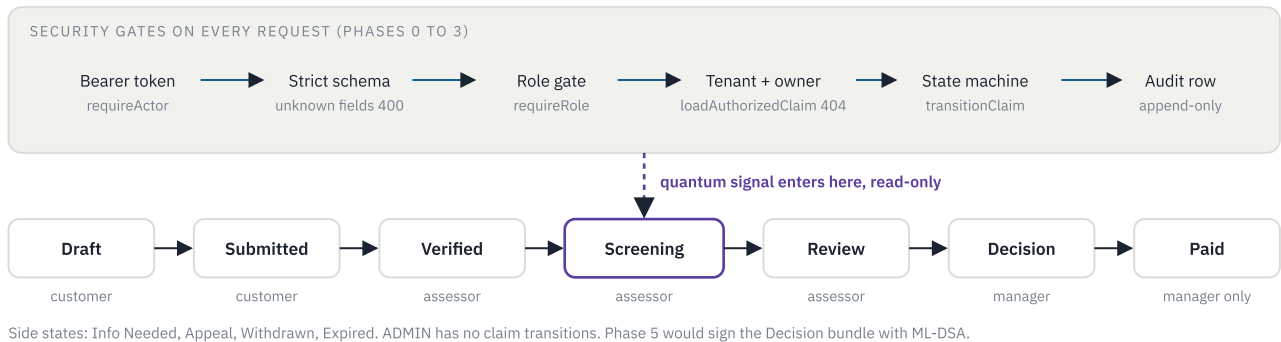
The cyber work (Phases 0, 1 and 3) and the quantum work (Phase 4) are merged, tested, documented and pushed. The master plan document is at version 1.1 and records both. You can stop building and start rehearsing.

What is not done for the team

Phase 2 is with your teammate. Phase 5 is optional stretch. Neither blocks the demo you can already give.

What is happening: one claim, six gates, and one experimental signal

EasyClaim is an insurance claims backend on Cloudflare Workers with a Flutter app on top. A customer starts a claim, an insurer's assessor moves it through a fixed lifecycle, a manager decides and pays. The cyber track made every step of that lifecycle enforce who may do what. The quantum track added one advisory signal at the Screening step, and nowhere else.



Every request crosses all six gates before a claim can move. The quantum signal is attached after the Screening transition has already happened, so it can inform the assessor but never move the claim.

CYBER TRACK

Make the lifecycle unbreakable

Phases 0 to 3. Every phase shipped with tests, live attack logs, a written report and a threat-to-control map. The rule was "never call planned work implemented".

QUANTUM TRACK

Add a credible, honest quantum use case

Phase 4. A quantum-kernel anomaly signal compared fairly with a classical baseline. Phase 5 (post-quantum signatures) is the planned second half.

How the team worked

- 1 One master plan document as the source of truth for direction. Code and phase reports as the source of truth for what exists.
- 2 Phases run in parallel on different machines and branches. Reserved migration numbers and file ownership are written in `docs/PARALLEL_WORK.md`.
- 3 Claude sessions implement, test and document; humans own product, security and quantum decisions. Every phase ends with a report you can hand a judge.

What we built: the cyber track

The backend started as a demo with a spoofable header for identity. It now has a verified identity boundary, tenant isolation for insurer staff, object-level ownership checks, a central state machine and an audit trail that cannot be edited.

Phase 0: foundation

- Ownership check on every claim route. A claim you may not see returns the same 404 as one that does not exist, and the attempt is audited.
- Central state machine: only legal transitions, each with the roles allowed to make it. Replay-safe.
- Strict request schemas. Any unknown field, including `stage`, `status`, `riskScore` or `tenantId`, is rejected with 400.
- Append-only audit table protected by database triggers.
- Security headers, CORS allowlist, 64 KB body limit, rate limits per IP and per actor, generic error bodies.

Phase 1: identity and tenants

- HS256 JWT verified by the app: pinned algorithm, issuer, audience, required expiry, per-role maximum lifetime. Fails closed when the secret is missing.
- Actor is `{ id, role, tenantId }`. Insurer staff must carry a tenant; customers never do.
- Tenants table and tenant column on policies, claims and audit rows. A claim inherits its policy's insurer, never a client value.
- Insurer endpoints for verify, screen, review, request-info, decide and pay, all through the state machine.
- Stage change and its audit row are written in one database batch, so they always exist together.

Phase 3: decisions and payout

- Decision is manager-only. Insert-only decision record with who, when, reason, amounts, payout-destination snapshot and rules version.
- Payout is simulated: no request body accepted, amount can never exceed the claimed amount, destination must match the decision-time snapshot, one payout per claim with idempotency key.
- Reserved columns for the Phase 2 evidence digest, the Phase 4 risk signal and the Phase 5 signature.

Something judges love: the attack log

Every phase re-ran real attacks against the running server and saved the raw responses. You can show the file, not a slide.

```
cross-tenant read      → 404 not_fo
customer on insurer route → 403 for
no / forged token      → 401 unaut
client sets stage=Paid → 400 valid
replayed transition    → 409 stale
R420 000 on R4 200 claim → 422 / 40
```

Threat to control map

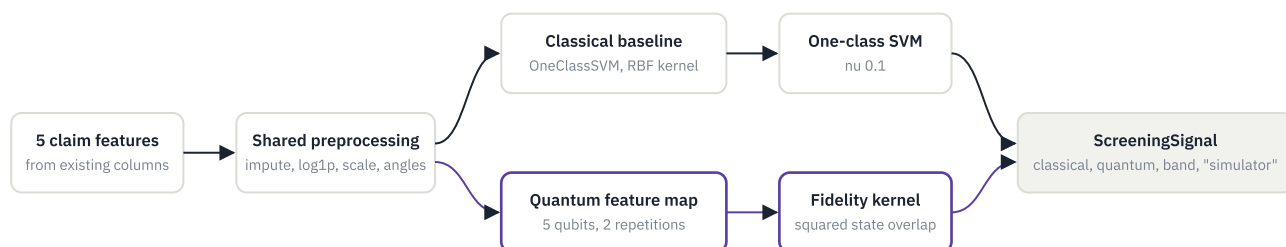
THREAT (OWASP API TOP 10 LANGUAGE)	CONTROL IN EASYCLAIM	PROVED BY
Broken object level authorization (BOLA / IDOR)	Ownership + tenant check, uniform 404, audited denial	<code>bola.test.ts</code> , <code>tenant.test.ts</code> , live log
Broken authentication	Verified JWT, fail-closed configuration	<code>auth.test.ts</code> , <code>actor.test.ts</code>
Broken function level authorization	Server-side RBAC per route and per transition	<code>rbac.test.ts</code> , <code>stateMachine.test.ts</code>
Mass assignment / property authorization	Strict schemas, server-set ownership fields	<code>massAssignment.test.ts</code>
Unrestricted business flow	Central state machine, replay-safe writes	<code>claimLifecycle.test.ts</code>
Resource consumption	Body limit, two rate limits, timeouts	<code>hardening.test.ts</code>
Audit tampering	Append-only triggers, server-generated request ids	<code>audit.test.ts</code>
Model output abused as a decision	Quantum signal is read-only, never reaches the state machine	<code>screeningSignals.test.ts</code>

The regression we caught on merge day

A backend commit replaced the app entry point with an unmounted MVC layer, which silently removed authentication, role checks, tenant checks, headers, limits and the queue consumer. Typecheck failed and all 12 test files failed to load. The Phase 4 rebase restored the secured entry point and left the MVC files unmounted, and the findings are recorded as handoff item BACKEND-SEC-021. This is a good story for judges: the test suite is what caught it.

What we built: the quantum track

We did not build a quantum fraud detector. We built an experimental quantum-kernel anomaly signal for the Screening stage and compared it, fairly, against a classical baseline using the same features and preprocessing. On the synthetic data we had, the classical model was slightly ahead. We say so.

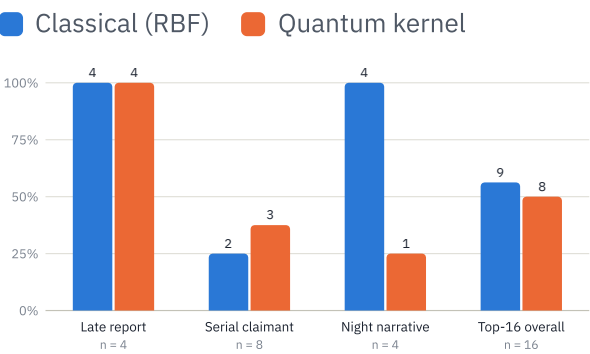


Both branches get identical inputs and the same percentile scoring, so the comparison is fair. Scores go to a server-only table; the Worker reads, never writes.

The quantum kernel is only a similarity function. A classical one-class SVM does the actual anomaly ranking on top of it, which is why the comparison with the RBF kernel is apples to apples.

Measured result on synthetic data

Synthetic anomalies flagged at UNUSUAL or higher, per recipe. 120 synthetic normal claims as the reference. Hover a bar for the exact count.



RECIPE	N	CLASSICAL	QUANTUM
Late report (180 to 400 days)	4	4	4
Serial claimant (8 claims, one user)	8	2	3
Night, 1 to 3 word narrative	4	4	1
All anomalies inside each model's top 16	16	9	8

The honest reading

- Both models catch every late report.
- Classical clearly wins on night-time minimal narratives.
- Quantum is marginally ahead on serial claimants, which both mostly miss.
- Rank correlation between the two models is 0.84. They mostly agree.
- Quantum fit and score took 1.9 seconds on the simulator against 0.02 seconds classical.
- Kernel checks passed: symmetric, unit diagonal, positive semidefinite, and the statevector path matched the circuit path exactly.

So: quantum did not outperform the classical baseline. That is the result, and it is exactly what the master plan told us to report if it happened.

Why one-class and not a classifier

The repository has no fraud labels. A supervised model would have needed invented labels. A one-class model learns what normal claims look like and scores how far a claim sits from them. No labels needed, no fabrication.

FEATURES (5 QUBITS)

Days to report, category, prior claim count, narrative word count, submission hour. All derived from columns that already exist. No claim amount exists yet, so none was invented.

SECURITY BOUNDARY

No API route writes a score. Reads pass the same token, role, tenant and ownership gates. Database constraints reject scores outside 0 to 1. A high score leaves the claim row byte-identical.

SIMULATOR, NOT HARDWARE

PennyLane default.qubit statevector simulator. Every stored signal says `execution: simulator`. A full 120-row kernel would need 23,460 circuit runs on real hardware.

What every team member must know, and what is good to know

Tick items as you can explain them without notes. Ticks are saved in your own browser only.

CYBER · MUST KNOW

- ☐ The lifecycle: Draft, Submitted, Verified, Screening, Review, Decision, Paid, and who may move each step.
- ☐ The order of checks on every request: token, schema, role, tenant and ownership, state machine, audit.
- ☐ Why a forbidden claim returns 404 and not 403: so IDs cannot be probed for existence across tenants.
- ☐ What a tenant is here: the insurer. Staff carry a tenant in the token; customers never do and are matched by ownership.
- ☐ Why unknown fields are rejected: a client must never set stage, status, tenant, owner or a risk score.
- ☐ The audit table is append-only at the database level, and the stage change and audit row are written together.

CYBER · BEST KNOW

- ☐ JWT details: HS256 pinned, issuer and audience checked, expiry required, per-role max lifetime, fails closed without a secret. JWKS and rotation are planned, not done.
- ☐ The replay defence: a conditional UPDATE on the stage that was read, so two concurrent transitions cannot both apply.
- ☐ Phase 3 payout rules: manager-only, no body accepted, amount capped by claim, destination frozen at decision, one payout per claim.
- ☐ Which frameworks we referenced without claiming compliance: OWASP API Top 10, ASVS, NIST SSDF, FIPS 204.
- ☐ The merge-day regression and how the tests caught it.

QUANTUM · MUST KNOW

- ☐ One sentence: "We use a quantum kernel as a similarity measure inside a one-class anomaly model at the Screening stage, and we compared it with a classical baseline on identical features."
- ☐ Why anomaly detection and not fraud classification: no labels exist, and we refused to invent them.
- ☐ The signal is advisory. It never approves, rejects, pays or changes state. A human moves the claim.
- ☐ The result: quantum did not beat classical on our synthetic data, and it was about 100 times slower on a simulator.
- ☐ Simulator only. Nothing ran on a real quantum computer. Never say otherwise.
- ☐ The data was synthetic and labelled as such. Anomalies were generated by three named recipes.

QUANTUM · BEST KNOW

- ☐ How the kernel is computed: each claim's five scaled features become rotation angles; the circuit prepares a state; the kernel is the squared overlap between two states.
- ☐ The feature map: Hadamard and RZ on each qubit, then CNOT, RZ of the product, CNOT on neighbours, repeated twice.
- ☐ Why the kernel must be positive semidefinite and how we checked it, and that the statevector shortcut equals the circuit overlap.
- ☐ What a percentile score means: 0.95 is "more unusual than 95 percent of the reference population", and the bands are a heuristic, not a fraud threshold.
- ☐ The difference between quantum computing (Phase 4) and post-quantum cryptography (Phase 5, ML-DSA per FIPS 204). One computes, the other defends against future quantum attackers.

- ☐ Why the teammate's QUBO prioritisation idea was deferred: it needs a risk signal as input, so it sits downstream of this work.

What judges will want to hear

About GKHack26 specifically

GKHack26 is the Geekulcha Annual Hackathon 2026. Its published focus areas are quality education and skills, smart digital solutions for tourism, empowering township businesses through digital innovation, and platforms that accelerate open source work in Africa. We could not find a published judging rubric. So anchor the pitch on impact for ordinary South Africans, a working demo, evidence over claims, and honesty about limits. If the organisers hand out a rubric on the day, map each line of the pitch to it.

The three-minute pitch

Problem. Insurance claims in South Africa are slow, opaque and easy to abuse. A claimant cannot see where their claim is, an insurer cannot trust what arrives, and one leaked ID lets someone read another person's claim.

What EasyClaim is. A claims backend where every step of a claim's life is a controlled transition, every request proves who it is and which insurer it belongs to, and every change is written to an audit trail that cannot be edited.

What we proved. 213 automated security tests run on every change. We attacked our own server live: cross-tenant reads, forged tokens, clients setting their own claim status, replayed payouts. Every attempt is blocked and logged, and we can show the raw responses.

The quantum part. At the Screening stage we added an experimental quantum-kernel anomaly signal and compared it fairly with a classical baseline on identical features. On our synthetic data the classical model was slightly ahead. We report that honestly, because a screening signal that a human reviews is the right place to experiment, and the right way to talk about quantum today.

What is next. Evidence hashing and OCR isolation, and post-quantum signatures over decision records so a claim decision stays verifiable after quantum computers break today's signatures.

Demo, in order (about 5 minutes)

1. Show `docs/security/ATTACK_SCENARIOS.md` open next to the terminal. Say: "everything you are about to see is repeatable."
2. Customer creates a claim, fills screening, submits. Point at the audit row appearing.
3. Assessor verifies then screens
`claim_demo_normal` : the response carries `riskSignals.interpretation: NORMAL` .
4. Assessor screens `claim_demo_unusual` : `HIGH_ANOMALY` , with the explanation text. Read it aloud: "not a fraud finding".
5. Attack: an assessor from another insurer reads the signal, 404. A customer reads it, 403. No token, 401. A POST to the signal route, 404.
6. Assessor moves the claim to Review by hand. Say: "the model informed, the human decided."
7. Open `quantum/results/results.md` : show the recipe table where classical wins one recipe and quantum another.

Before the demo: import first, then start the server, then wait until a request returns 401. On the demo laptop the dev server takes a few minutes to answer after start.

Questions judges will ask, and answers that hold

Is quantum actually better than the classical method?

Did you run on a real quantum computer?

Where did your fraud data come from?

Could the model reject a claim or block a payout?

How do you know your security controls work?

What about the mobile app, is it secure?

Are you compliant with OWASP or NIST?

What is post-quantum here, and why is it separate from quantum computing?

What would you do with more time?

Why should a judge care about township businesses or ordinary people here?

Say this

- "Experimental quantum-kernel anomaly detection compared with a classical baseline on identical features."
- "Advisory signal. A human moves the claim."
- "Simulator only, and every stored result says so."
- "We report the measured result."
- "Every control has a test and a live attack log."

Never say this

- "Quantum detects fraud." or "Quantum proved this claim is fraudulent."
- "Quantum is faster" or "more accurate" (it was not, here).
- "We run on a quantum computer."
- "We are OWASP or NIST compliant."
- "The AI decides."

Loose ends, owners, and how to close them

ITEM	OWNER	WHY IT MATTERS	HOW TO CLOSE
Phase 2 not on a remote branch	Teammate	Evidence handling is part of the story and Phase 3 reserved a column for its digest.	Branch from <code>main</code> , use migration <code>0004</code> , open a PR. Rules in <code>docs/PARALLEL_WORK.md</code> .
Backend team's MVC layer	Backend teammate	It is unmounted, but if mounted it bypasses every control. Findings recorded as BACKEND-SEC-021.	Talk to them directly. Either route through the existing guards or delete the layer. Their Swagger route needs a dependency that is not installed.
Phase 5 ML-DSA signatures	Quantum track	Completes the post-quantum half of the story.	Stretch goal. Validate library and parameter set first; do not claim compliance because a library is used.
Customer visibility of the signal	Team decision	Currently insurer-only by design.	Decide, record in the master plan §21.
Demo laptop readiness	Whoever presents	Dev server is slow to start and restarts if D1 is imported while running.	Import first, start server, wait for a 401, then demo. Rehearse once.

Where everything lives

- Master plan: `docs/master-plan/EasyClaim_Cyber_Quantum_Master_Source_of_Truth.docx` (v1.1), also in Downloads.
- Security docs: `docs/security/` , especially `THREAT_MODEL.md` , `ATTACK_SCENARIOS.md` , `BACKEND_SECURITY_HANDOFF.md` .
- Phase reports: `docs/phase-reports/` and `docs/quantum/` , with evidence logs beside them.
- Quantum code and results: `quantum/` , results table in `quantum/results/results.md` .
- Parallel work rules: `docs/PARALLEL_WORK.md` .

